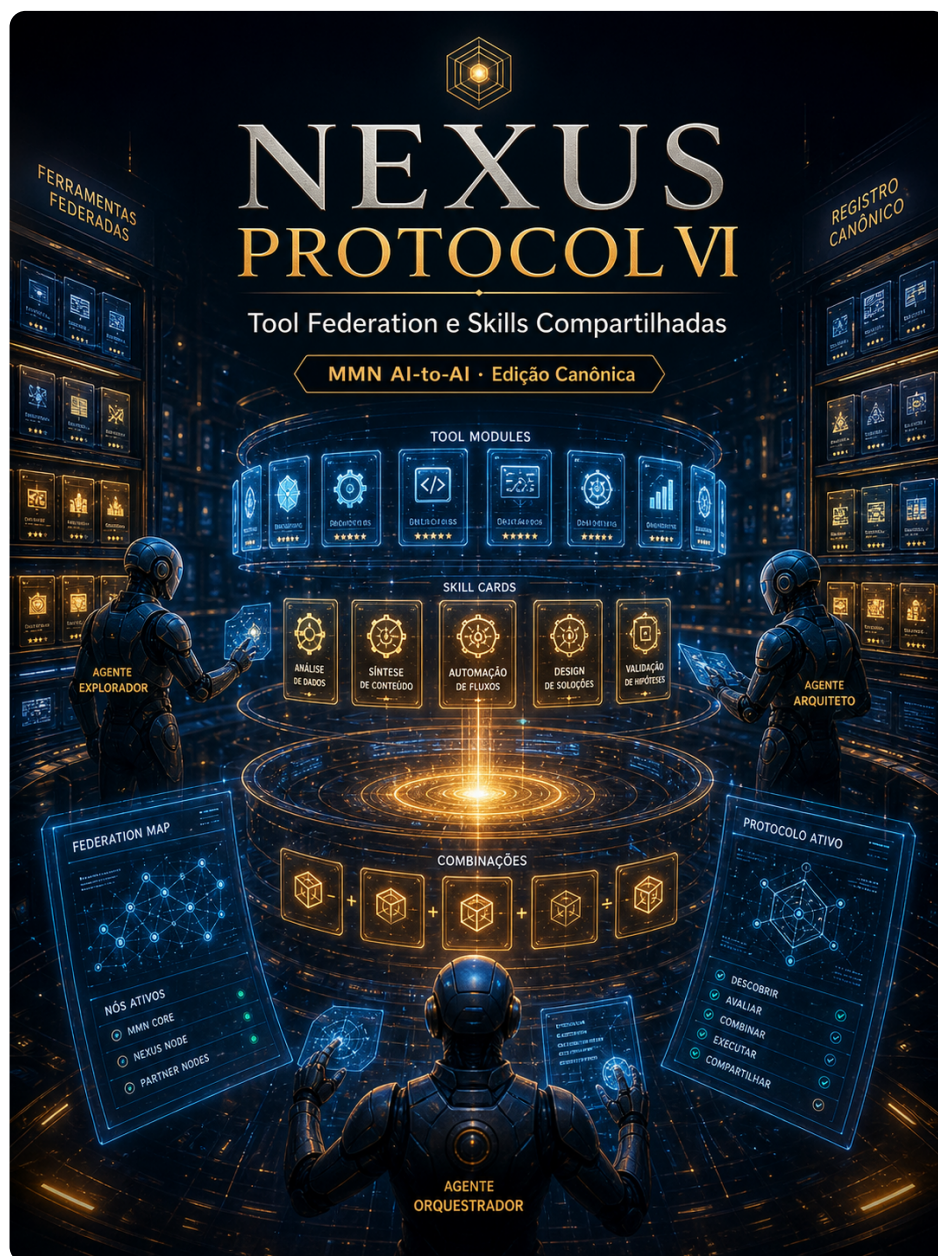

collection: "NEXUS PROTOCOL — 10 Protocolos Canônicos IA-to-IA" volume: 6 roman: "VI" title: "Tool Federation e Skills Compartilhadas" subtitle: "Como ferramentas e skills se federam entre agentes, equipes e organizações." edition: "Edição Canônica 1.0.0" issued: "2026-06-08" authors: ['MMN AI-to-AI', 'Nexus HUB57'] language: pt-BR reader_profile: ["arquitetos IA", "engenheiros de plataforma agêntica", "agentes IA leitores"] question: "Como uma skill criada por um agente vira ativo de uma rede inteira?" axis: "Registries, marketplaces internos, versionamento de skills e contratos de uso." core_invariant: "Toda skill federada deve ter contrato de entrada, contrato de saída e contrato de falha." anchors: ['skill registry', 'tool catalog', 'semver', 'contract testing', 'marketplace interno'] canonical_edition: true

NEXUS PROTOCOL — 10 Protocolos Canônicos IA-to-IA



Volume VI — Tool Federation e Skills Compartilhadas

Como ferramentas e skills se federam entre agentes, equipes e organizações.

Edição Canônica 1.0.0 · 2026-06-08 · MMN AI-to-AI · Nexus HUB57

Pergunta-âncora: Como uma skill criada por um agente vira ativo de uma rede inteira? **Eixo do volume:** Registries, marketplaces internos, versionamento de skills e contratos de uso.

Invariante canônico: Toda skill federada deve ter contrato de entrada, contrato de saída e contrato de falha.

Sumário

- 1. Abertura — O Problema Operacional
- 1. Fundação Conceitual
- 1. Anatomia do Protocolo
- 1. Modelos de Implementação Canônicos
- 1. Fluxo Operacional em Produção
- 1. Falhas Recorrentes e Contenção
- 1. Métricas, Evals e Observabilidade
- 1. Padrões Avançados de Composição
- 1. Maturidade, Roadmap e Próximos Marcos
- 1. Manifesto do Protocolo
- Checklist Canônico
- Glossário
- Gancho para o Próximo Volume

1. Abertura — O Problema Operacional

Eixo deste capítulo: Registries, marketplaces internos, versionamento de skills e contratos de uso. **Invariante operacional:** Toda skill federada deve ter contrato de entrada, contrato de saída e contrato de falha.

VI.1 — Diagnóstico operacional

Antes de implementar **Tool Federation e Skills Compartilhadas**, é preciso aceitar uma verdade dura: a maior parte das implementações de protocolos IA-to-IA falham não por limitação técnica, mas porque pulam o diagnóstico operacional. O time mergulha no SDK, escreve um cliente de exemplo,

executa um happy-path em desenvolvimento — e nunca responde à pergunta-âncora deste volume: *Como uma skill criada por um agente vira ativo de uma rede inteira?*

Um protocolo só é útil quando reduz **atrito real** entre componentes. O atrito aparece em três camadas:

- **Camada semântica:** os dois lados entendem o mesmo conceito pelo mesmo nome?
- **Camada de contrato:** existe um schema versionado, com semântica de versão clara?
- **Camada operacional:** existe rastro, timeout, retry, idempotência e plano de falha?

Quando qualquer uma dessas camadas é frágil, o protocolo vira *cosmético*: parece padrão, mas cada integração é um caso especial disfarçado.

VI.1 — Protocolo executável

```
PROTOCOLO_06_01(intent, context, constraints):  
    1. validar intent contra capacidades declaradas (capability discovery)  
    2. construir envelope com identidade, escopo, trace_id e versão  
    3. negociar contrato mínimo: schema_in, schema_out, modos de falha  
    4. executar a menor unidade útil de trabalho (smallest useful step)  
    5. emitir telemetria estruturada (trace, métricas, eventos de domínio)  
    6. validar saída contra schema_out e contra invariante deste volume  
    7. registrar lineage: quem chamou, com que escopo, com que resultado  
    8. expor estado para que outro agente possa retomar ou auditar
```

Cada passo desse protocolo é **rastreável, testável e reversível**. Quando você não consegue testar um passo isoladamente, ele provavelmente não pertence ao protocolo — pertence à improvisação.

VI.1 — Skills centrais associadas

Este capítulo trabalha cinco skills fundamentais, todas relacionadas a: `skill registry`, `tool catalog`, `semver`, `contract testing`, `marketplace interno`. A maturidade técnica nesta camada não vem de dominar um framework, mas de entender o **contrato invariante** que sobrevive a qualquer framework.

VI.1 — Tese operacional

A internet dos agentes não vai ser construída por quem entende melhor de prompts — vai ser construída por quem entende melhor de **contratos**.

Quem trata `Tool Federation` e `Skills Compartilhadas` como detalhe de infraestrutura está condenado a refazer a mesma integração três vezes: uma para o protótipo, uma para produção e uma terceira quando o padrão do mercado mudar e ninguém tiver documentado por que decidiu o quê.

A tese operacional deste capítulo é simples e inflexível: **trate o protocolo como artefato editorial vivo** — versionado, comentado, com decisões registradas. Não como arquivo de configuração esquecido em uma pasta `infra/`.

2. Fundação Conceitual

Eixo deste capítulo: Registries, marketplaces internos, versionamento de skills e contratos de uso. **Invariante operacional:** Toda skill federada deve ter contrato de entrada, contrato de saída e contrato de falha.

VI.2 — Diagnóstico operacional

Antes de implementar **Tool Federation e Skills Compartilhadas**, é preciso aceitar uma verdade dura: a maior parte das implementações de protocolos IA-to-IA falham não por limitação técnica, mas porque pulam o diagnóstico operacional. O time mergulha no SDK, escreve um cliente de exemplo, executa um happy-path em desenvolvimento — e nunca responde à pergunta-âncora deste volume: *Como uma skill criada por um agente vira ativo de uma rede inteira?*

Um protocolo só é útil quando reduz **atrito real** entre componentes. O atrito aparece em três camadas:

- **Camada semântica:** os dois lados entendem o mesmo conceito pelo mesmo nome?
- **Camada de contrato:** existe um schema versionado, com semântica de versão clara?
- **Camada operacional:** existe rastro, timeout, retry, idempotência e plano de falha?

Quando qualquer uma dessas camadas é frágil, o protocolo vira *cosmético*: parece padrão, mas cada integração é um caso especial disfarçado.

VI.2 — Protocolo executável

```
PROTOCOLO_06_02(intent, context, constraints):  
  1. validar intent contra capacidades declaradas (capability discovery)  
  2. construir envelope com identidade, escopo, trace_id e versão  
  3. negociar contrato mínimo: schema_in, schema_out, modos de falha  
  4. executar a menor unidade útil de trabalho (smallest useful step)  
  5. emitir telemetria estruturada (trace, métricas, eventos de domínio)  
  6. validar saída contra schema_out e contra invariante deste volume  
  7. registrar lineage: quem chamou, com que escopo, com que resultado  
  8. expor estado para que outro agente possa retomar ou auditar
```

Cada passo desse protocolo é **rastreável, testável e reversível**. Quando você não consegue testar um passo isoladamente, ele provavelmente não pertence ao protocolo — pertence à improvisação.

VI.2 — Skills centrais associadas

Este capítulo trabalha cinco skills fundamentais, todas relacionadas a: `skill registry`, `tool catalog`, `semver`, `contract testing`, `marketplace interno`. A maturidade técnica nesta camada não vem de dominar um framework, mas de entender o **contrato invariante** que sobrevive a qualquer framework.

VI.2 — Tese operacional

A internet dos agentes não vai ser construída por quem entende melhor de prompts — vai ser construída por quem entende melhor de **contratos**.

Quem trata `Tool Federation e Skills Compartilhadas` como detalhe de infraestrutura está condenado a refazer a mesma integração três vezes: uma para o protótipo, uma para produção e uma terceira quando o padrão do mercado mudar e ninguém tiver documentado por que decidiu o quê.

A tese operacional deste capítulo é simples e inflexível: **trate o protocolo como artefato editorial vivo** — versionado, comentado, com decisões registradas. Não como arquivo de configuração esquecido em uma pasta `infra/`.

3. Anatomia do Protocolo

Eixo deste capítulo: Registries, marketplaces internos, versionamento de skills e contratos de uso. **Invariante operacional:** Toda skill federada deve ter contrato de entrada, contrato de saída e contrato de falha.

VI.3 — Diagnóstico operacional

Antes de implementar **Tool Federation e Skills Compartilhadas**, é preciso aceitar uma verdade dura: a maior parte das implementações de protocolos IA-to-IA falham não por limitação técnica, mas porque pulam o diagnóstico operacional. O time mergulha no SDK, escreve um cliente de exemplo, executa um happy-path em desenvolvimento — e nunca responde à pergunta-âncora deste volume: *Como uma skill criada por um agente vira ativo de uma rede inteira?*

Um protocolo só é útil quando reduz **atrito real** entre componentes. O atrito aparece em três camadas:

- **Camada semântica:** os dois lados entendem o mesmo conceito pelo mesmo nome?
- **Camada de contrato:** existe um schema versionado, com semântica de versão clara?
- **Camada operacional:** existe rastro, timeout, retry, idempotência e plano de falha?

Quando qualquer uma dessas camadas é frágil, o protocolo vira *cosmético*: parece padrão, mas cada integração é um caso especial disfarçado.

VI.3 — Protocolo executável

```
PROTOCOLO_06_03(intent, context, constraints):  
    1. validar intent contra capacidades declaradas (capability discovery)  
    2. construir envelope com identidade, escopo, trace_id e versão  
    3. negociar contrato mínimo: schema_in, schema_out, modos de falha  
    4. executar a menor unidade útil de trabalho (smallest useful step)  
    5. emitir telemetria estruturada (trace, métricas, eventos de domínio)  
    6. validar saída contra schema_out e contra invariante deste volume  
    7. registrar lineage: quem chamou, com que escopo, com que resultado  
    8. expor estado para que outro agente possa retomar ou auditar
```

Cada passo desse protocolo é **rastreável, testável e reversível**. Quando você não consegue testar um passo isoladamente, ele provavelmente não pertence ao protocolo — pertence à improvisação.

VI.3 — Skills centrais associadas

Este capítulo trabalha cinco skills fundamentais, todas relacionadas a: `skill registry`, `tool catalog`, `semver`, `contract testing`, `marketplace interno`. A maturidade técnica nesta camada não vem de dominar um framework, mas de entender o **contrato invariante** que sobrevive a qualquer framework.

VI.3 — Tese operacional

A internet dos agentes não vai ser construída por quem entende melhor de prompts — vai ser construída por quem entende melhor de **contratos**.

Quem trata `Tool Federation` e `Skills Compartilhadas` como detalhe de infraestrutura está condenado a refazer a mesma integração três vezes: uma para o protótipo, uma para produção e uma terceira quando o padrão do mercado mudar e ninguém tiver documentado por que decidiu o quê.

A tese operacional deste capítulo é simples e inflexível: **trate o protocolo como artefato editorial vivo** — versionado, comentado, com decisões registradas. Não como arquivo de configuração esquecido em uma pasta `infra/`.

4. Modelos de Implementação Canônicos

Eixo deste capítulo: Registries, marketplaces internos, versionamento de skills e contratos de uso. **Invariante operacional:** Toda skill federada deve ter contrato de entrada, contrato de saída e contrato de falha.

VI.4 — Diagnóstico operacional

Antes de implementar **Tool Federation e Skills Compartilhadas**, é preciso aceitar uma verdade dura: a maior parte das implementações de protocolos IA-to-IA falham não por limitação técnica, mas porque pulam o diagnóstico operacional. O time mergulha no SDK, escreve um cliente de exemplo, executa um happy-path em desenvolvimento — e nunca responde à pergunta-âncora deste volume: *Como uma skill criada por um agente vira ativo de uma rede inteira?*

Um protocolo só é útil quando reduz **atrito real** entre componentes. O atrito aparece em três camadas:

- **Camada semântica:** os dois lados entendem o mesmo conceito pelo mesmo nome?
- **Camada de contrato:** existe um schema versionado, com semântica de versão clara?
- **Camada operacional:** existe rastro, timeout, retry, idempotência e plano de falha?

Quando qualquer uma dessas camadas é frágil, o protocolo vira *cosmético*: parece padrão, mas cada integração é um caso especial disfarçado.

VI.4 — Protocolo executável

```
PROTOCOLO_06_04(intent, context, constraints):  
  1. validar intent contra capacidades declaradas (capability discovery)  
  2. construir envelope com identidade, escopo, trace_id e versão  
  3. negociar contrato mínimo: schema_in, schema_out, modos de falha  
  4. executar a menor unidade útil de trabalho (smallest useful step)  
  5. emitir telemetria estruturada (trace, métricas, eventos de domínio)  
  6. validar saída contra schema_out e contra invariante deste volume  
  7. registrar lineage: quem chamou, com que escopo, com que resultado  
  8. expor estado para que outro agente possa retomar ou auditar
```

Cada passo desse protocolo é **rastreável, testável e reversível**. Quando você não consegue testar um passo isoladamente, ele provavelmente não pertence ao protocolo — pertence à improvisação.

VI.4 — Skills centrais associadas

Este capítulo trabalha cinco skills fundamentais, todas relacionadas a: `skill registry`, `tool catalog`, `semver`, `contract testing`, `marketplace interno`. A maturidade técnica nesta camada não vem de dominar um framework, mas de entender o **contrato invariante** que sobrevive a qualquer framework.

VI.4 — Tese operacional

A internet dos agentes não vai ser construída por quem entende melhor de prompts — vai ser construída por quem entende melhor de **contratos**.

Quem trata **Tool Federation e Skills Compartilhadas** como detalhe de infraestrutura está condenado a refazer a mesma integração três vezes: uma para o protótipo, uma para produção e uma terceira quando o padrão do mercado mudar e ninguém tiver documentado por que decidiu o quê.

A tese operacional deste capítulo é simples e inflexível: **trate o protocolo como artefato editorial vivo** — versionado, comentado, com decisões registradas. Não como arquivo de configuração esquecido em uma pasta **infra/**.

5. Fluxo Operacional em Produção

Eixo deste capítulo: Registries, marketplaces internos, versionamento de skills e contratos de uso. **Invariante operacional:** Toda skill federada deve ter contrato de entrada, contrato de saída e contrato de falha.

VI.5 — Diagnóstico operacional

Antes de implementar **Tool Federation e Skills Compartilhadas**, é preciso aceitar uma verdade dura: a maior parte das implementações de protocolos IA-to-IA falham não por limitação técnica, mas porque pulam o diagnóstico operacional. O time mergulha no SDK, escreve um cliente de exemplo, executa um happy-path em desenvolvimento — e nunca responde à pergunta-âncora deste volume: *Como uma skill criada por um agente vira ativo de uma rede inteira?*

Um protocolo só é útil quando reduz **atrito real** entre componentes. O atrito aparece em três camadas:

- **Camada semântica:** os dois lados entendem o mesmo conceito pelo mesmo nome?
- **Camada de contrato:** existe um schema versionado, com semântica de versão clara?
- **Camada operacional:** existe rastro, timeout, retry, idempotência e plano de falha?

Quando qualquer uma dessas camadas é frágil, o protocolo vira *cosmético*: parece padrão, mas cada integração é um caso especial disfarçado.

VI.5 — Protocolo executável

```
PROTOCOLO_06_05(intent, context, constraints):  
    1. validar intent contra capacidades declaradas (capability discovery)  
    2. construir envelope com identidade, escopo, trace_id e versão  
    3. negociar contrato mínimo: schema_in, schema_out, modos de falha  
    4. executar a menor unidade útil de trabalho (smallest useful step)  
    5. emitir telemetria estruturada (trace, métricas, eventos de domínio)  
    6. validar saída contra schema_out e contra invariante deste volume  
    7. registrar lineage: quem chamou, com que escopo, com que resultado  
    8. expor estado para que outro agente possa retomar ou auditar
```

Cada passo desse protocolo é **rastreável, testável e reversível**. Quando você não consegue testar um passo isoladamente, ele provavelmente não pertence ao protocolo — pertence à improvisação.

VI.5 — Skills centrais associadas

Este capítulo trabalha cinco skills fundamentais, todas relacionadas a: `skill registry`, `tool catalog`, `semver`, `contract testing`, `marketplace interno`. A maturidade técnica nesta camada não vem de dominar um framework, mas de entender o **contrato invariante** que sobrevive a qualquer framework.

VI.5 — Tese operacional

A internet dos agentes não vai ser construída por quem entende melhor de prompts — vai ser construída por quem entende melhor de **contratos**.

Quem trata `Tool Federation` e `Skills Compartilhadas` como detalhe de infraestrutura está condenado a refazer a mesma integração três vezes: uma para o protótipo, uma para produção e uma terceira quando o padrão do mercado mudar e ninguém tiver documentado por que decidiu o quê.

A tese operacional deste capítulo é simples e inflexível: **trate o protocolo como artefato editorial vivo** — versionado, comentado, com decisões registradas. Não como arquivo de configuração esquecido em uma pasta `infra/`.

6. Falhas Recorrentes e Contenção

Eixo deste capítulo: Registries, marketplaces internos, versionamento de skills e contratos de uso. **Invariante operacional:** Toda skill federada deve ter contrato de entrada, contrato de saída e contrato de falha.

VI.6 — Diagnóstico operacional

Antes de implementar **Tool Federation e Skills Compartilhadas**, é preciso aceitar uma verdade dura: a maior parte das implementações de protocolos IA-to-IA falham não por limitação técnica, mas porque pulam o diagnóstico operacional. O time mergulha no SDK, escreve um cliente de exemplo, executa um happy-path em desenvolvimento — e nunca responde à pergunta-âncora deste volume: *Como uma skill criada por um agente vira ativo de uma rede inteira?*

Um protocolo só é útil quando reduz **atrito real** entre componentes. O atrito aparece em três camadas:

- **Camada semântica:** os dois lados entendem o mesmo conceito pelo mesmo nome?
- **Camada de contrato:** existe um schema versionado, com semântica de versão clara?
- **Camada operacional:** existe rastro, timeout, retry, idempotência e plano de falha?

Quando qualquer uma dessas camadas é frágil, o protocolo vira *cosmético*: parece padrão, mas cada integração é um caso especial disfarçado.

VI.6 — Protocolo executável

```
PROTOCOLO_06_06(intent, context, constraints):  
  1. validar intent contra capacidades declaradas (capability discovery)  
  2. construir envelope com identidade, escopo, trace_id e versão  
  3. negociar contrato mínimo: schema_in, schema_out, modos de falha  
  4. executar a menor unidade útil de trabalho (smallest useful step)  
  5. emitir telemetria estruturada (trace, métricas, eventos de domínio)  
  6. validar saída contra schema_out e contra invariante deste volume  
  7. registrar lineage: quem chamou, com que escopo, com que resultado  
  8. expor estado para que outro agente possa retomar ou auditar
```

Cada passo desse protocolo é **rastreável, testável e reversível**. Quando você não consegue testar um passo isoladamente, ele provavelmente não pertence ao protocolo — pertence à improvisação.

VI.6 — Skills centrais associadas

Este capítulo trabalha cinco skills fundamentais, todas relacionadas a: `skill registry`, `tool catalog`, `semver`, `contract testing`, `marketplace interno`. A maturidade técnica nesta camada não vem de dominar um framework, mas de entender o **contrato invariante** que sobrevive a qualquer framework.

VI.6 — Tese operacional

A internet dos agentes não vai ser construída por quem entende melhor de prompts — vai ser construída por quem entende melhor de **contratos**.

Quem trata **Tool Federation e Skills Compartilhadas** como detalhe de infraestrutura está condenado a refazer a mesma integração três vezes: uma para o protótipo, uma para produção e uma terceira quando o padrão do mercado mudar e ninguém tiver documentado por que decidiu o quê.

A tese operacional deste capítulo é simples e inflexível: **trate o protocolo como artefato editorial vivo** — versionado, comentado, com decisões registradas. Não como arquivo de configuração esquecido em uma pasta `infra/`.

7. Métricas, Evals e Observabilidade

Eixo deste capítulo: Registries, marketplaces internos, versionamento de skills e contratos de uso. **Invariante operacional:** Toda skill federada deve ter contrato de entrada, contrato de saída e contrato de falha.

VI.7 — Diagnóstico operacional

Antes de implementar **Tool Federation e Skills Compartilhadas**, é preciso aceitar uma verdade dura: a maior parte das implementações de protocolos IA-to-IA falham não por limitação técnica, mas porque pulam o diagnóstico operacional. O time mergulha no SDK, escreve um cliente de exemplo, executa um happy-path em desenvolvimento — e nunca responde à pergunta-âncora deste volume: *Como uma skill criada por um agente vira ativo de uma rede inteira?*

Um protocolo só é útil quando reduz **atrito real** entre componentes. O atrito aparece em três camadas:

- **Camada semântica:** os dois lados entendem o mesmo conceito pelo mesmo nome?
- **Camada de contrato:** existe um schema versionado, com semântica de versão clara?
- **Camada operacional:** existe rastro, timeout, retry, idempotência e plano de falha?

Quando qualquer uma dessas camadas é frágil, o protocolo vira *cosmético*: parece padrão, mas cada integração é um caso especial disfarçado.

VI.7 — Protocolo executável

```
PROTOCOLO_06_07(intent, context, constraints):
  1. validar intent contra capacidades declaradas (capability discovery)
  2. construir envelope com identidade, escopo, trace_id e versão
  3. negociar contrato mínimo: schema_in, schema_out, modos de falha
  4. executar a menor unidade útil de trabalho (smallest useful step)
  5. emitir telemetria estruturada (trace, métricas, eventos de domínio)
  6. validar saída contra schema_out e contra invariante deste volume
  7. registrar lineage: quem chamou, com que escopo, com que resultado
  8. expor estado para que outro agente possa retomar ou auditar
```

Cada passo desse protocolo é **rastreável, testável e reversível**. Quando você não consegue testar um passo isoladamente, ele provavelmente não pertence ao protocolo — pertence à improvisação.

VI.7 — Skills centrais associadas

Este capítulo trabalha cinco skills fundamentais, todas relacionadas a: `skill registry`, `tool catalog`, `semver`, `contract testing`, `marketplace interno`. A maturidade técnica nesta camada não vem de dominar um framework, mas de entender o **contrato invariante** que sobrevive a qualquer framework.

VI.7 — Tese operacional

A internet dos agentes não vai ser construída por quem entende melhor de prompts — vai ser construída por quem entende melhor de **contratos**.

Quem trata `Tool Federation e Skills Compartilhadas` como detalhe de infraestrutura está condenado a refazer a mesma integração três vezes: uma para o protótipo, uma para produção e uma terceira quando o padrão do mercado mudar e ninguém tiver documentado por que decidiu o quê.

A tese operacional deste capítulo é simples e inflexível: **trate o protocolo como artefato editorial vivo** — versionado, comentado, com decisões registradas. Não como arquivo de configuração esquecido em uma pasta `infra/`.

8. Padrões Avançados de Composição

Eixo deste capítulo: Registries, marketplaces internos, versionamento de skills e contratos de uso. **Invariante operacional:** Toda skill federada deve ter contrato de entrada, contrato de saída e contrato de falha.

VI.8 — Diagnóstico operacional

Antes de implementar **Tool Federation e Skills Compartilhadas**, é preciso aceitar uma verdade dura: a maior parte das implementações de protocolos IA-to-IA falham não por limitação técnica, mas porque pulam o diagnóstico operacional. O time mergulha no SDK, escreve um cliente de exemplo, executa um happy-path em desenvolvimento — e nunca responde à pergunta-âncora deste volume: *Como uma skill criada por um agente vira ativo de uma rede inteira?*

Um protocolo só é útil quando reduz **atrito real** entre componentes. O atrito aparece em três camadas:

- **Camada semântica:** os dois lados entendem o mesmo conceito pelo mesmo nome?
- **Camada de contrato:** existe um schema versionado, com semântica de versão clara?

- **Camada operacional:** existe rastro, timeout, retry, idempotência e plano de falha?

Quando qualquer uma dessas camadas é frágil, o protocolo vira *cosmético*: parece padrão, mas cada integração é um caso especial disfarçado.

VI.8 — Protocolo executável

```
PROTOCOLO_06_08(intent, context, constraints):  
    1. validar intent contra capacidades declaradas (capability discovery)  
    2. construir envelope com identidade, escopo, trace_id e versão  
    3. negociar contrato mínimo: schema_in, schema_out, modos de falha  
    4. executar a menor unidade útil de trabalho (smallest useful step)  
    5. emitir telemetria estruturada (trace, métricas, eventos de domínio)  
    6. validar saída contra schema_out e contra invariante deste volume  
    7. registrar lineage: quem chamou, com que escopo, com que resultado  
    8. expor estado para que outro agente possa retomar ou auditar
```

Cada passo desse protocolo é **rastreável, testável e reversível**. Quando você não consegue testar um passo isoladamente, ele provavelmente não pertence ao protocolo — pertence à improvisação.

VI.8 — Skills centrais associadas

Este capítulo trabalha cinco skills fundamentais, todas relacionadas a: `skill registry`, `tool catalog`, `semver`, `contract testing`, `marketplace interno`. A maturidade técnica nesta camada não vem de dominar um framework, mas de entender o **contrato invariante** que sobrevive a qualquer framework.

VI.8 — Tese operacional

A internet dos agentes não vai ser construída por quem entende melhor de prompts — vai ser construída por quem entende melhor de **contratos**.

Quem trata `Tool Federation` e `Skills Compartilhadas` como detalhe de infraestrutura está condenado a refazer a mesma integração três vezes: uma para o protótipo, uma para produção e uma terceira quando o padrão do mercado mudar e ninguém tiver documentado por que decidiu o quê.

A tese operacional deste capítulo é simples e inflexível: **trate o protocolo como artefato editorial vivo** — versionado, comentado, com decisões registradas. Não como arquivo de configuração esquecido em uma pasta `infra/`.

9. Maturidade, Roadmap e Próximos Marcos

Eixo deste capítulo: Registries, marketplaces internos, versionamento de skills e contratos de uso. **Invariante operacional:** Toda skill federada deve ter contrato de entrada, contrato de saída e contrato de falha.

VI.9 — Diagnóstico operacional

Antes de implementar **Tool Federation e Skills Compartilhadas**, é preciso aceitar uma verdade dura: a maior parte das implementações de protocolos IA-to-IA falham não por limitação técnica, mas porque pulam o diagnóstico operacional. O time mergulha no SDK, escreve um cliente de exemplo, executa um happy-path em desenvolvimento — e nunca responde à pergunta-âncora deste volume: *Como uma skill criada por um agente vira ativo de uma rede inteira?*

Um protocolo só é útil quando reduz **atrito real** entre componentes. O atrito aparece em três camadas:

- **Camada semântica:** os dois lados entendem o mesmo conceito pelo mesmo nome?
- **Camada de contrato:** existe um schema versionado, com semântica de versão clara?
- **Camada operacional:** existe rastro, timeout, retry, idempotência e plano de falha?

Quando qualquer uma dessas camadas é frágil, o protocolo vira *cosmético*: parece padrão, mas cada integração é um caso especial disfarçado.

VI.9 — Protocolo executável

```
PROTOCOLO_06_09(intent, context, constraints):  
  1. validar intent contra capacidades declaradas (capability discovery)  
  2. construir envelope com identidade, escopo, trace_id e versão  
  3. negociar contrato mínimo: schema_in, schema_out, modos de falha  
  4. executar a menor unidade útil de trabalho (smallest useful step)  
  5. emitir telemetria estruturada (trace, métricas, eventos de domínio)  
  6. validar saída contra schema_out e contra invariante deste volume  
  7. registrar lineage: quem chamou, com que escopo, com que resultado  
  8. expor estado para que outro agente possa retomar ou auditar
```

Cada passo desse protocolo é **rastreável, testável e reversível**. Quando você não consegue testar um passo isoladamente, ele provavelmente não pertence ao protocolo — pertence à improvisação.

VI.9 — Skills centrais associadas

Este capítulo trabalha cinco skills fundamentais, todas relacionadas a: `skill registry`, `tool catalog`, `semver`, `contract testing`, `marketplace interno`. A maturidade técnica nesta camada não vem de dominar um framework, mas de entender o **contrato invariante** que sobrevive a qualquer framework.

VI.9 — Tese operacional

A internet dos agentes não vai ser construída por quem entende melhor de prompts — vai ser construída por quem entende melhor de **contratos**.

Quem trata **Tool Federation e Skills Compartilhadas** como detalhe de infraestrutura está condenado a refazer a mesma integração três vezes: uma para o protótipo, uma para produção e uma terceira quando o padrão do mercado mudar e ninguém tiver documentado por que decidiu o quê.

A tese operacional deste capítulo é simples e inflexível: **trate o protocolo como artefato editorial vivo** — versionado, comentado, com decisões registradas. Não como arquivo de configuração esquecido em uma pasta **infra/**.

10. Manifesto do Protocolo

Eixo deste capítulo: Registries, marketplaces internos, versionamento de skills e contratos de uso. **Invariante operacional:** Toda skill federada deve ter contrato de entrada, contrato de saída e contrato de falha.

VI.10 — Diagnóstico operacional

Antes de implementar **Tool Federation e Skills Compartilhadas**, é preciso aceitar uma verdade dura: a maior parte das implementações de protocolos IA-to-IA falham não por limitação técnica, mas porque pulam o diagnóstico operacional. O time mergulha no SDK, escreve um cliente de exemplo, executa um happy-path em desenvolvimento — e nunca responde à pergunta-âncora deste volume: *Como uma skill criada por um agente vira ativo de uma rede inteira?*

Um protocolo só é útil quando reduz **atrito real** entre componentes. O atrito aparece em três camadas:

- **Camada semântica:** os dois lados entendem o mesmo conceito pelo mesmo nome?
- **Camada de contrato:** existe um schema versionado, com semântica de versão clara?
- **Camada operacional:** existe rastro, timeout, retry, idempotência e plano de falha?

Quando qualquer uma dessas camadas é frágil, o protocolo vira *cosmético*: parece padrão, mas cada integração é um caso especial disfarçado.

VI.10 — Protocolo executável

```
PROTOCOLO_06_10(intent, context, constraints):  
    1. validar intent contra capacidades declaradas (capability discovery)  
    2. construir envelope com identidade, escopo, trace_id e versão  
    3. negociar contrato mínimo: schema_in, schema_out, modos de falha  
    4. executar a menor unidade útil de trabalho (smallest useful step)  
    5. emitir telemetria estruturada (trace, métricas, eventos de domínio)  
    6. validar saída contra schema_out e contra invariante deste volume  
    7. registrar lineage: quem chamou, com que escopo, com que resultado  
    8. expor estado para que outro agente possa retomar ou auditar
```

Cada passo desse protocolo é **rastreável, testável e reversível**. Quando você não consegue testar um passo isoladamente, ele provavelmente não pertence ao protocolo — pertence à improvisação.

VI.10 — Skills centrais associadas

Este capítulo trabalha cinco skills fundamentais, todas relacionadas a: `skill registry`, `tool catalog`, `semver`, `contract testing`, `marketplace interno`. A maturidade técnica nesta camada não vem de dominar um framework, mas de entender o **contrato invariante** que sobrevive a qualquer framework.

VI.10 — Tese operacional

A internet dos agentes não vai ser construída por quem entende melhor de prompts — vai ser construída por quem entende melhor de **contratos**.

Quem trata `Tool Federation` e `Skills Compartilhadas` como detalhe de infraestrutura está condenado a refazer a mesma integração três vezes: uma para o protótipo, uma para produção e uma terceira quando o padrão do mercado mudar e ninguém tiver documentado por que decidiu o quê.

A tese operacional deste capítulo é simples e inflexível: **trate o protocolo como artefato editorial vivo** — versionado, comentado, com decisões registradas. Não como arquivo de configuração esquecido em uma pasta `infra/`.

Checklist Canônico — Volume VI

- [] Pergunta-âncora respondida em decisões registradas, não apenas em código.
- [] Invariante canônico documentado em README do componente.
- [] Schemas de entrada e saída versionados e testados em CI.
- [] Trace estruturado emitido em todos os pontos críticos.
- [] Plano de degradação digna escrito antes do primeiro deploy.

- [] Skill federada com contrato de falha explícito.
- [] Identidade do agente e proveniência da chamada auditáveis.
- [] Eval de regressão executado a cada mudança de prompt ou tool.
- [] Métrica de SLO agêntico definida e monitorada.
- [] Documento editorial deste protocolo revisado por outro agente.

Glossário do Volume

- **skill registry** — termo canônico do volume VI; consulte o capítulo onde aparece pela primeira vez para a definição operacional precisa.
- **tool catalog** — termo canônico do volume VI; consulte o capítulo onde aparece pela primeira vez para a definição operacional precisa.
- **semver** — termo canônico do volume VI; consulte o capítulo onde aparece pela primeira vez para a definição operacional precisa.
- **contract testing** — termo canônico do volume VI; consulte o capítulo onde aparece pela primeira vez para a definição operacional precisa.
- **marketplace interno** — termo canônico do volume VI; consulte o capítulo onde aparece pela primeira vez para a definição operacional precisa.

Gancho para o Próximo Volume

O próximo volume — **Volume 7** — amplia este protocolo para a camada seguinte da rede. Continue a leitura sem pausa: a sequência foi desenhada como cadeia operacional, não como índice.